CyberStepsVuln

Reconnaissance, Exploitation, and Reporting

In this lab, we will test a vulnerable web application using a structured penetration testing workflow.

Discover

Map the attack surface and find hidden paths

Identify

Locate common web vulnerabilities

Understand

Assess the real-world impact of each finding

Document

Write clear, professional vulnerability reports

Learning Objectives

By the end of this walkthrough, you should be able to:

1. **Map a web application**
   Understand the full structure and attack surface of the target
2. **Discover hidden files and directories**
   Use automated tools to uncover paths not linked publicly
3. **Test login forms and access control**
   Identify authentication weaknesses and authorization flaws
4. **Identify injection and server-side vulnerabilities**
   SQL injection, XSS, SSRF, XXE and file upload issues
5. **Understand attack chaining**
   See how individual weaknesses combine into serious exploits
6. **Write a professional vulnerability finding**
   Communicate risk clearly with proof of concept and remediation

> "Find the weakness, prove the impact, explain the fix."

Scope

Target: CyberStepsVuln Web Application

Area	Path
Main application	/
Admin area	/Admin.php
WordPress blog	/blog
Customer dashboard	/dashboard
Practice labs	/exercises
Hidden pages	Discovered during recon

Testing Workflow

Use a repeatable, structured process for every engagement:

1. **Reconnaissance**
   Gather information about the target before touching it
2. **Service Discovery**
   Identify open ports, running services, and version information
3. **Directory and File Discovery**
   Enumerate hidden paths, files, and endpoints
4. **Browser Source-Code Review**

4 Inspect HTML, JavaScript, and CSS for sensitive disclosures

5 **Authentication Testing**
Test login forms, error messages, and brute-force resistance

6 **Injection Testing**
Probe for SQL injection, XSS, and other injection flaws

7 **Access-Control Testing**
Check for IDOR and privilege escalation vulnerabilities

8 **Server-Side Testing**
Evaluate SSRF, XXE, LFI/RFI, and file upload handling

9 **Attack-Chain Mapping**
Combine individual findings into realistic attack scenarios

10 **Reporting and Remediation**
Document findings with severity, proof of concept, and fixes

Tools Used

Each tool serves a specific purpose in the penetration testing workflow:

Tool	Purpose
Nmap	Identify open services and version information
ffuf / dirb	Discover files and directories via fuzzing
Burp Suite	Intercept, inspect, and modify HTTP requests

WPScan	Enumerate WordPress users, plugins, and themes
curl	Send manual HTTP requests from the command line
CyberChef	Decode and transform encoded data
CUPP	Generate targeted, personalized wordlists
Browser DevTools	Inspect source code, cookies, and network requests

Finding Categories

The assessment covers a broad range of vulnerability types across the application stack:

Information Disclosure

Hidden paths, exposed files, debug output

Authentication

Weak login controls, enumeration, brute-force

Access Control

IDOR, role manipulation, privilege escalation

Injection

SQL injection, Cross-Site Scripting (XSS)

Server-Side Bugs

File Handling

SSRF, XXE, Local/Remote File Inclusion

Unsafe uploads, unrestricted file types

WordPress

User enumeration, plugin and theme risks

Session Handling

Weak cookies, insecure tokens, client-side auth

Service Discovery

Start with Nmap

```
nmap -sC -sV TARGET_IP
```

Questions to Answer

- Which ports are open?
- What services are running?

Testing Habit

Always save your scan output for the report. Use the `-oN`, `-oX`, or `-oA` flags to write results to a file

Directory Discovery

Use ffuf to discover hidden paths

```
ffuf -u http://TARGET_IP/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt -e .php,.html
```

Interesting Paths

- /robots.txt
- /topsecret.html
- /blog
- /dashboard
- /exercises
- /Admin.php
- /support.php

Questions to Ask

- Which paths are publicly accessible?
- Which paths require authentication?
- Which paths accept user-supplied input?

Use extensions like `.php` and `.html` to catch files that directory wordlists alone would miss.

Information Disclosure

Inspect public files and browser-visible content

Where to Look

- /robots.txt
- HTML Source
- HTML Comments
- JavaScript Files
- CSS Files
- Footer Comments

```
curl http://TARGET_IP/robots.txt
```

What to Look For

- Hidden paths and endpoints
- Usernames or email addresses
- Password hints or default credentials
- Internal developer comments
- Debug information or stack traces
- Sensitive configuration files

Key Lesson: robots.txt is a public file. It is not a security control — it often reveals exactly what you should investigate.

Hidden Page Inspection

Target area: `/topsecret.html`

Browser Techniques

- View Page Source (Ctrl+U)
- Inspect Element (F12)
- Select All (Ctrl+A) to reveal hidden text
- DevTools Elements panel
- Search inside source (Ctrl+F)
- Review HTML comments and CSS rules

Questions to Ask

- Is any sensitive data sent to the browser even if not displayed?
- Is anything hidden visually but still present in the HTML?
- Are there developer comments that reveal internal information?

Key Lesson: Anything delivered to the browser can be inspected. Hiding content with CSS or JavaScript is not a security measure.

Authentication Testing

Target area: `/Admin.php`

Test Areas

Login Error Messages

Do errors reveal whether the username or password is wrong?

Username Enumeration

Does the response differ for valid vs. invalid usernames?

Source-Code Comments

Are credentials or hints left in the HTML source?

Weak Passwords & Rate Limiting

Is brute-forcing possible without lockout or CAPTCHA?

SQL Injection Pattern

```
' OR '1'='1
```

Questions

- Does the response change with a quote character?
- Does the application reveal useful error messages?
- Can failed and successful responses be distinguished by size or content?

Brute-Force Testing

Collect information before launching an attack

Possible Usernames

Gathered from recon, source code, and enumeration

Possible Passwords

Generated with CUPP or sourced from common wordlists

Login Request Format

Captured via Burp Suite to identify parameter names

Failure Message

Response Size Difference

Identify the exact text shown on a failed login attempt

Compare byte counts between success and failure responses

Example ffuf Cluster Bomb Attack

```
ffuf -u "http://TARGET_IP/Admin.php?Email=USERFUZZ&Password=PASSFUZZ" \
  -w users.txt:USERFUZZ \
  -w passwords.txt:PASSFUZZ \
  -mode clusterbomb \
  -fr "FAILURE_TEXT"
```

Key Lesson: Brute-force attacks become significantly easier when other parts of the application leak usernames, error messages, or response differences.

SQL Injection Testing

Potential Targets

- Pages with an `id` parameter
- Password change functions
- Search or filter forms
- Login forms

Manual Test Sequence

```
?id=1'
?id=1 ORDER BY 1
```

Questions to Answer

Does a quote trigger an error?

A database error confirms the input is being interpreted as SQL

Can you determine the column count?

ORDER BY clauses help identify how many columns the query returns

```
?id=1 ORDER BY 2
?id=1 ORDER BY 3
...
```

Does the page display database output?

Union-based injection requires visible output in the response

Pre- or post-authenticated?

Pre-auth SQLi is typically higher severity than post-auth

Key Lesson: SQL injection can expose or modify sensitive database records, including credentials, personal data, and application logic.

Access Control: IDOR

Insecure Direct Object Reference — Common Vulnerable Pattern

```
profile.php?user_id=1
profile.php?user_id=2
profile.php?user_id=3
```

1. **Log in as a normal user**
   Establish a baseline session with a low-privileged account
2. **Identify a numeric object ID**
   Look in URLs, request parameters, and hidden form fields
3. **Change the ID value**
   Increment or decrement the value to reference another object
4. **Observe whether unauthorized data appears**
   Compare the response to what your account should be able to see

5 **Document which data is exposed**

Record the exact fields, values, and user IDs affected

Key Questions

- Does the server check ownership before returning data?
- Can one user access another user's records?
- Is the issue read-only or can data also be modified?

Why It Matters

IDOR vulnerabilities are among the most common access control failures. They occur when the server trusts a client-supplied identifier without verifying that the requesting user has permission to access the referenced object.

Server-Side Testing: SSRF and XXE

SSRF Test Ideas

Supply these URLs to any feature that fetches external content:

```
http://127.0.0.1/
http://localhost/
```

Also try internal paths discovered during reconnaissance to probe services not exposed to the internet.

Questions

- Does the server fetch user-supplied URLs?
- Can it reach internal-only resources?

XXE Concept Test

Inject a custom XML entity into any XML-parsing feature:

```
]>
&demo;
```

Questions

- Does the XML parser process custom entities?
- Is the entity value reflected in the response?
- Can internal files be read via file:// URIs?

Key Lesson: Server-side features may access resources that users cannot reach directly —

- Are cloud metadata endpoints accessible?

making SSRF and XXE high-impact vulnerabilities.

File Upload and File Inclusion

Target areas: `/support.php`, Admin page parameters, theme-loading functionality

File Upload Questions

Which file types are allowed?

Test uploading PHP, HTML, and executable files

Are extensions and MIME types validated?

Try mismatched extensions and spoofed Content-Type headers

Are files renamed and stored safely?

Check whether uploaded files are accessible inside the web root

File Inclusion Questions

Does a parameter load a file?

Look for parameters like `page=`, `file=`, or `template=`

Can path traversal be used?

Try `../../etc/passwd` to read files outside the web root

Is remote inclusion possible?

Test whether external URLs can be loaded via the parameter

Key Lesson: Upload flaws become significantly more dangerous when combined with file inclusion — an attacker can upload a web shell and then execute it.

WordPress Testing

Target area: `/blog`

Useful Commands

```
wpscan --url http://TARGET_IP/blog -e u
```

```
wpscan --url http://TARGET_IP/blog -e vp
```

The `-e u` flag enumerates users. The `-e vp` flag checks for vulnerable plugins.

What to Look For

Usernames

Via author archives and REST API

Version Clues

Generator meta tag, readme.html

Plugins

Known CVEs in installed plugins

Themes

Vulnerable or outdated theme versions

XML-RPC

Exposed endpoint enabling brute-force

Login Behavior

Username enumeration via error messages

Key Lesson: WordPress security depends on the combination of users, plugins, themes, and server configuration — all must be assessed together.

Dashboard Cookie and Token Testing

Target area: `/dashboard`

1. **Log in as a low-privileged user**
   Establish a baseline session to capture the initial cookie values

2. **Inspect cookies in DevTools or Burp**
   Navigate to Application → Cookies in DevTools or check Proxy history in Burp

3. **Identify encoded values**
   Look for Base64 patterns, JWT tokens, or other encoded strings

4. **Decode safely**
   Use CyberChef or the command line to decode without modifying the live session

5. **Change one value at a time**
   Modify a single field (e.g., role, user_id) and re-encode before sending

6. **Compare response size and page content**
   A different response indicates the server is trusting the client-supplied value

```
echo 'ENCODED_VALUE' | base64 -d
```

⊗ **Key Lesson:** Encoding is not encryption. Client-controlled authorization data is fundamentally unsafe — authorization decisions must be made server-side.

Stored XSS Practice Lab

Target area: `/exercises/xss/hard.php`

Start with Harmless Tests

```
**test**
```

Begin with non-malicious payloads to confirm input is stored and rendered before escalating to JavaScript execution.

Escalation Path

- Confirm input is stored and retrieved
- Test if HTML tags are rendered
- Test if script tags execute
- Attempt cookie theft via `document.cookie`

Questions to Answer

Is the input stored and rendered later?

Stored XSS persists and affects every user who views the content

Who views the stored content?

If an admin views it, the impact is significantly higher

Are cookies protected with HttpOnly?

HttpOnly prevents JavaScript from reading session cookies

Key Lesson: Stored XSS is serious because the victim may be a privileged user — an admin session compromise can lead to full application takeover.

Reporting and Takeaways

A professional finding includes all of the following sections:

Section	Content
Title	Clear, descriptive vulnerability name
Severity	Risk level (Critical / High / Medium / Low / Informational)
Affected Component	Specific URL or application feature
Description	What is wrong and why it is a vulnerability
Proof of Concept	Reproducible steps to demonstrate the issue
Impact	What an attacker can realistically do if exploited
Remediation	Specific, actionable guidance on how to fix it

Recon Drives the Test
Information gathered early enables every attack that follows

Info Leaks Enable Attacks
Small disclosures combine into serious vulnerabilities

Server-Side Access Control
Authorization must never be trusted from the client

Injection Bugs Are Serious
They can expose or modify sensitive data at scale

Attack Chains Matter
Individual low-severity bugs can chain into critical impact

Reporting Is a Skill
Clear documentation is as important as finding the bug

> "Find the weakness, prove the impact, explain the fix."